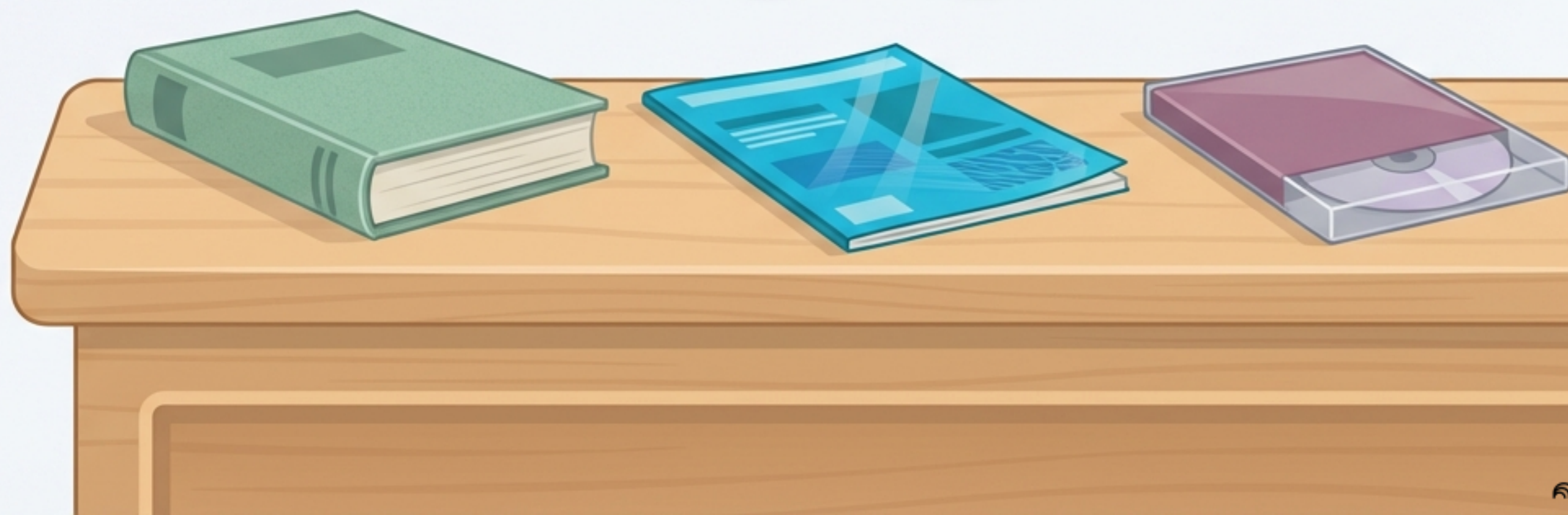
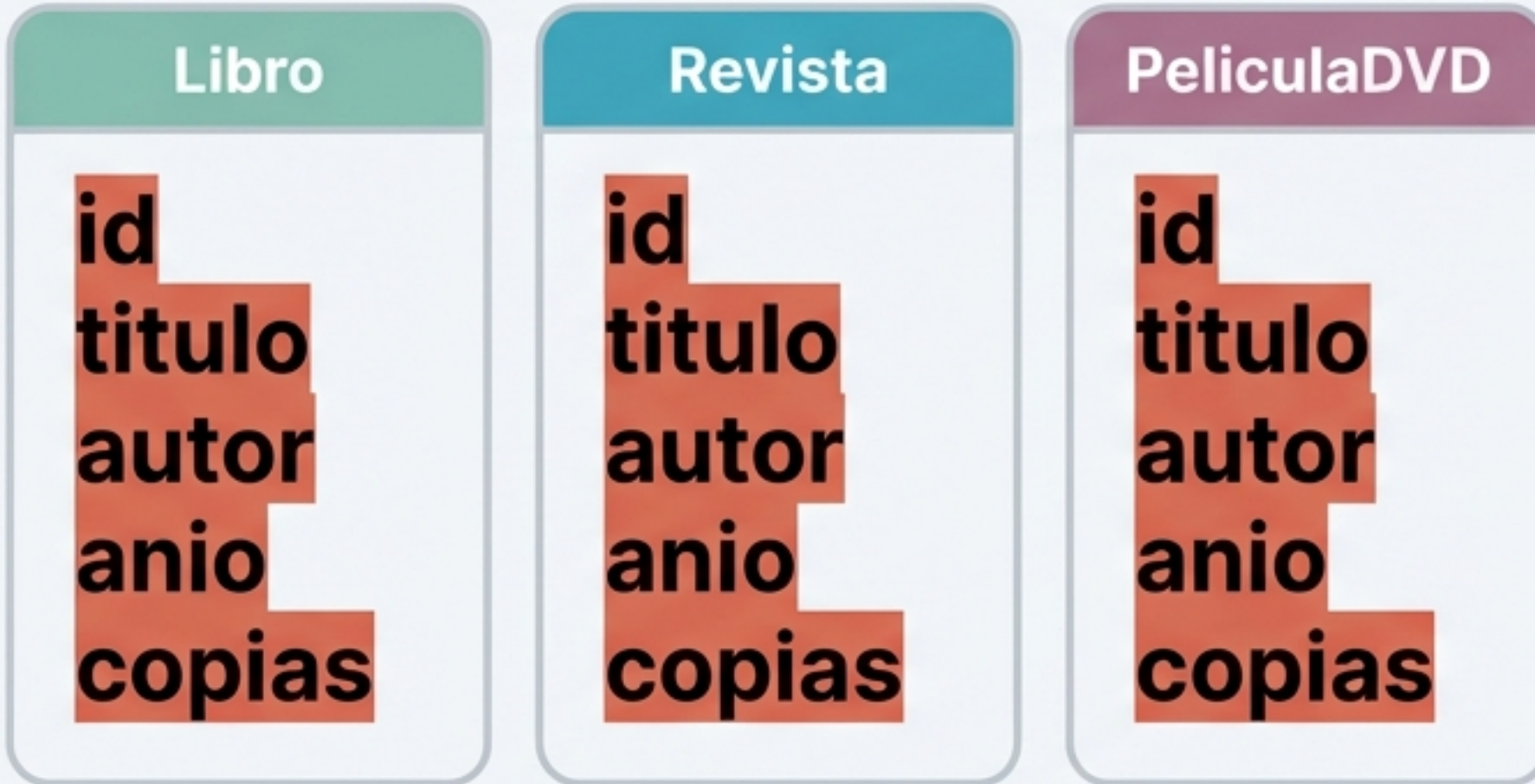


Bienvenido a la biblioteca de Kotlin

Sistema de Gestión de Biblioteca (Versión Inicial). Aprenderemos Programación Orientada a Objetos creando jerarquías de clases para catálogos reales.

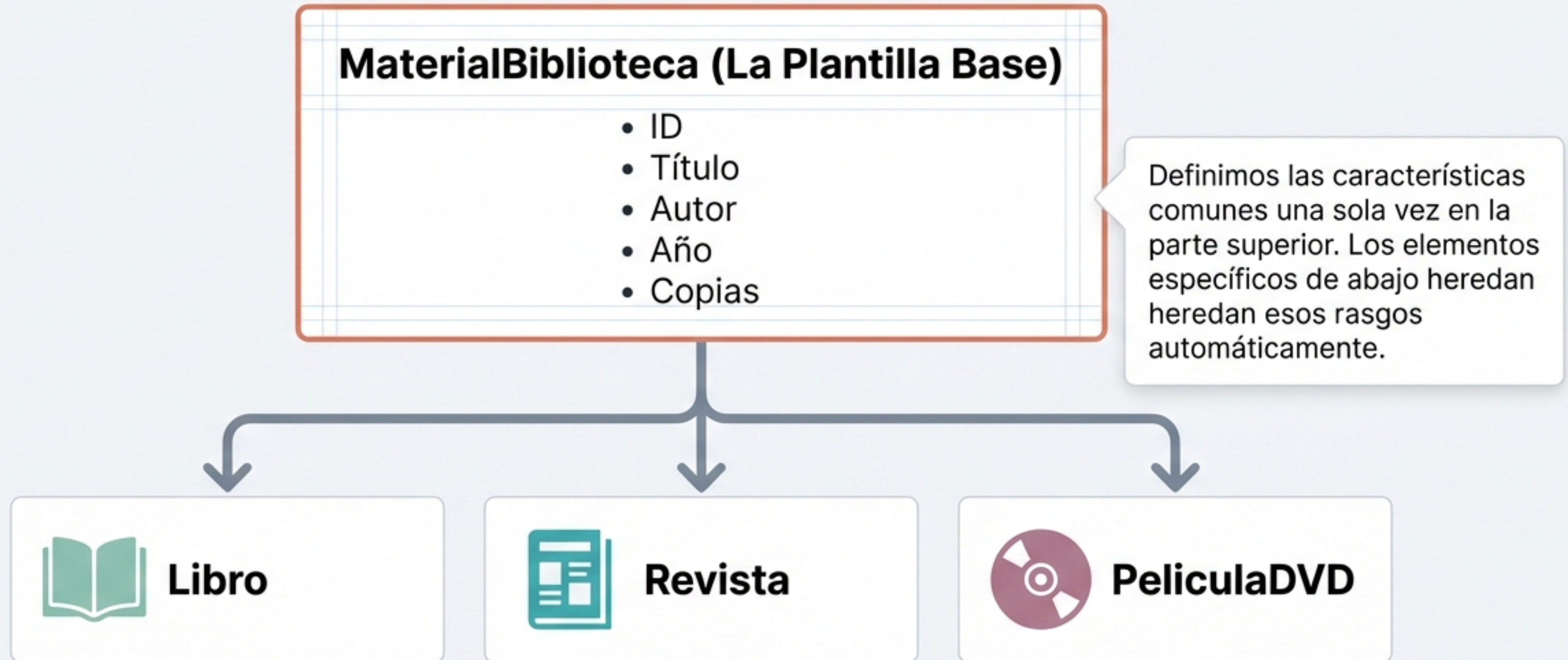


Escribir el mismo código repetidas veces genera errores

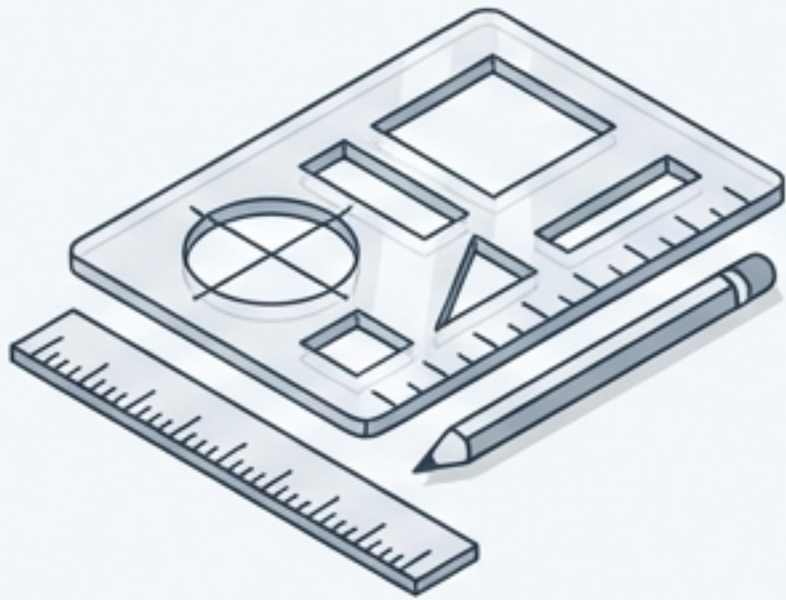


Si programamos cada material de forma independiente, repetimos las mismas variables básicas una y otra vez. **¿Cómo lo hacemos más inteligente?**

La jerarquía resuelve el problema de la repetición



Una clase abstracta define el molde, pero no se puede fabricar



```
abstract class MaterialBiblioteca(...)
```

Al usar la palabra clave `abstract`, indicamos que esta clase es solo una plantilla. No puedes crear un “Material” genérico en la vida real, solo puedes instanciar a sus hijos concretos (un libro físico o un DVD).

El constructor principal establece las propiedades universales

Indica que estas propiedades son de solo lectura una vez creadas.

```
abstract class MaterialBiblioteca(val id:String,  
    val titulo:String, val autor:String,  
    val anio:Int, val copias:Int)
```

Estos cinco datos existirán en cualquier objeto de la biblioteca, sin importar su formato.

El operador de dos puntos conecta a la clase hija con su padre

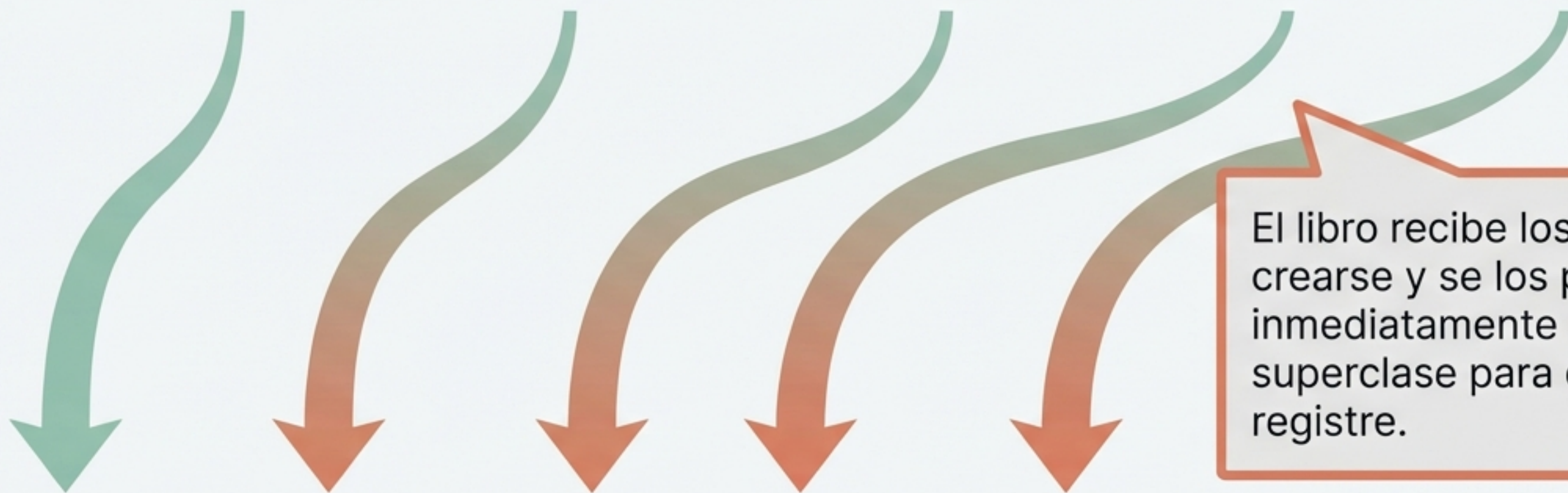


```
class Libro(...) : MaterialBiblioteca(...)
```

En Kotlin, los dos puntos (:) significan "**es un tipo de**". Aquí le decimos al sistema que Libro hereda todo de MaterialBiblioteca.

Los datos fluyen desde el hijo hacia el constructor del padre

```
class Libro(id:String, titulo:String, autor:String, anio:Int, copias:Int)
```



```
: MaterialBiblioteca(id, titulo, autor, anio, copias)
```

El libro recibe los datos al crearse y se los pasa inmediatamente a la superclase para que los registre.

La herencia permite añadir características exclusivas



```
class Libro(  
    id:String, titulo:String, autor:String, anio:Int, copias:Int,  
    numeroPaginas:Int  
) : MaterialBiblioteca(id, titulo, autor, anio, copias)
```



```
class Revista(  
    id:String, titulo:String, autor:String, anio:Int, copias:Int,  
    numeroEdicion:Int  
) : MaterialBiblioteca(id, titulo, autor, anio, copias)
```

Aunque comparten el título y el autor, un libro necesita contar sus páginas (**numeroPaginas**) y una revista necesita identificar su volumen (**numeroEdicion**).

Cada formato amplía la plantilla con sus propias reglas

```
class PeliculaDVD(  
    id:String, titulo:String, autor:String, anio:Int, copias:Int,  
    duracionMinutos:Int  
) : MaterialBiblioteca(id, titulo, autor, anio, copias)
```



El DVD añade la variable de tiempo. La herencia nos da un estándar básico pero nos permite flexibilidad total para lo específico.

La función override personaliza nuestros objetos

```
override fun toString():String {  
    return("$id | $titulo | $autor | $anio | $copias")  
}
```

Usamos override para **sobrescribir el comportamiento** por defecto. Ahora, al imprimir cualquier material, veremos sus detalles separados por una barra vertical.

El polimorfismo agrupa objetos diferentes bajo una misma identidad

```
inventario = mutableListOf<MaterialBiblioteca>()
```



Gracias al polimorfismo, no necesitamos tres listas separadas. Como todos "son un" `MaterialBiblioteca`, podemos guardarlos en la misma colección e iterar sobre ellos con un solo bucle `for`.

El encapsulamiento protege la integridad de los datos

```
private val inventario = mutableListOf<MaterialBiblioteca>()
```



Al hacer la lista privada, evitamos que otras partes del código la borren o modifiquen por accidente. Solo la función oficial **registrarMaterial** tiene la llave para añadir elementos.

```
fun registrarMaterial(material: MaterialBiblioteca) {  
    inventario.add(material)  
}
```


Los bucles interactivos dan vida al gestor

```
while(continuar) {  
    1. Inventario: Mostrar  
    2. Inventario: Registrar  
}
```

```
val entrada = readln().trim()
```

Un bucle infinito **while(true)** gestiona el menú interactivo, utilizando **trim()** para limpiar los espacios en blanco de las entradas del usuario y evitar registros vacíos.

Tres pilares para estructurar código escalable en Kotlin



Clases Abstractas

Plantillas base (abstract class) que definen características comunes pero no se instancian.



Herencia

Clases hijas que adoptan propiedades con el operador : y añaden las suyas propias.

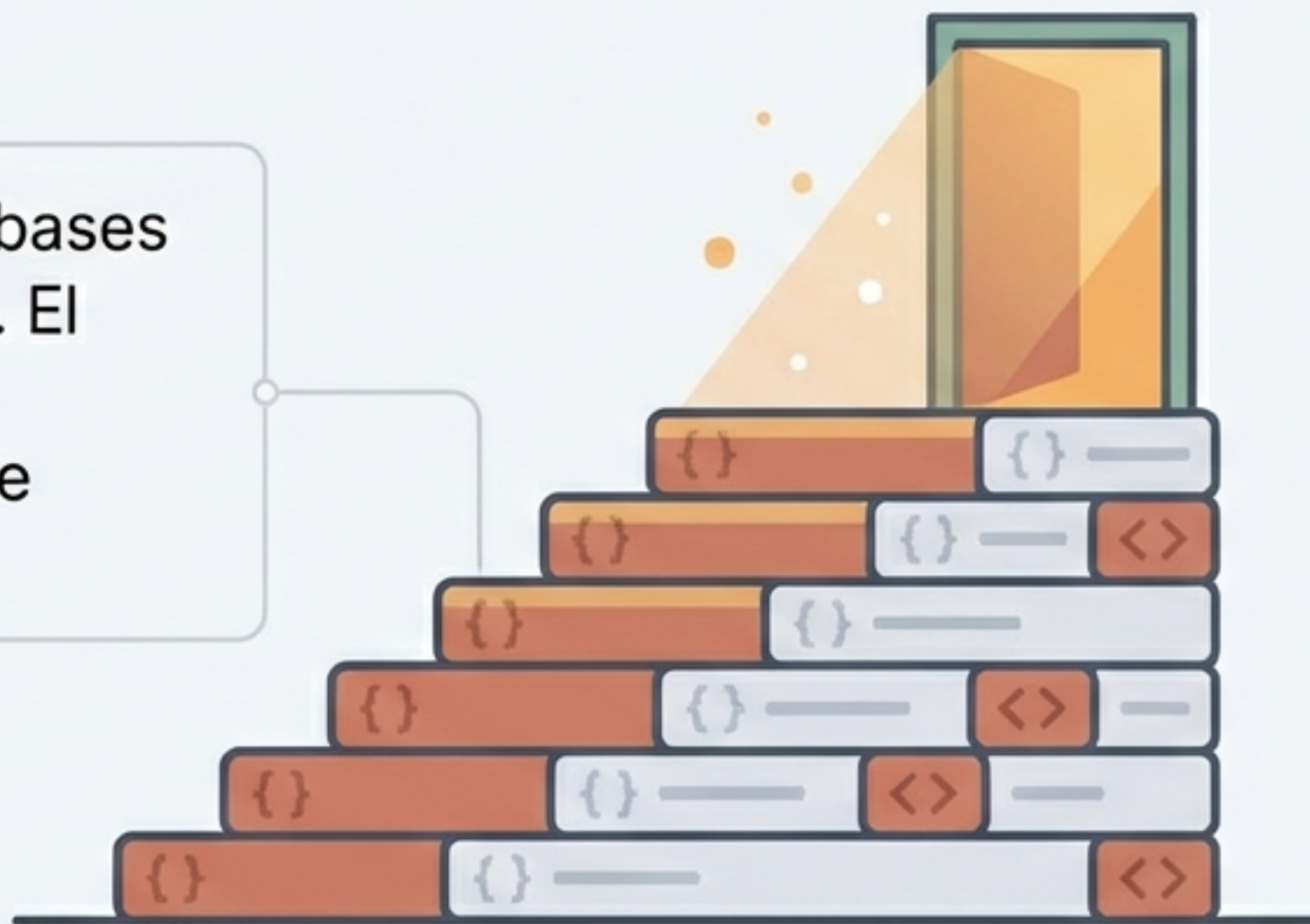


Polimorfismo

Capacidad de agrupar y manejar distintos objetos como si fueran un solo tipo general.

Tu sistema orientado a objetos está listo para crecer

Acabas de dominar las bases de la herencia en Kotlin. El código es limpio, sin repeticiones y altamente escalable.



Como desafío, intenta crear una nueva clase AudioLibro que herede de MaterialBiblioteca y añada la propiedad narrador. ¡El código ya está preparado para aceptarlo!